**EMSI – École Marocaine des Sciences de l'Ingénieur**
**Casablanca, Maroc**
**www.emsi.ma**

**ONEE – Branch Électricité**
**Direction du Système d'Information**
**Casablanca, Maroc**

# RAPPORT DE STAGE

## de Fin d'Année

## 4ème Année Ingénieur d'État

# Conception et Développement d'un Système de Gestion des Ressources Internes

*(Réservation des Salles de Réunion & Gestion du Parc Automobile)*

| | |
|---|---|
| **Réalisé par :** | Adamovic |
| **Filière :** | Intelligence Artificielle & Data Science |
| **Niveau :** | 4ème Année Ingénieur d'État |
| **Organisme d'accueil :** | ONEE – Branch Électricité, Direction du Système d'Information (DSI) |
| **Encadrant professionnel :** | M. Mouaad EL MASLOUHI – Responsable DSI |
| **Encadrant académique :** | M. Zakaria KHATAR – Professeur EMSI |
| **Période de stage :** | Juin – Août 2026 (10 semaines) |
| **Année Universitaire :** | 2025 – 2026 |

**Année Universitaire 2025 – 2026**

# Dédicace

*Je dédie ce travail à mes **parents**, dont le soutien indéfectible et l'amour inconditionnel ont été ma source d'énergie tout au long de ce parcours académique.*

*À mes **frères et sœurs**, pour leur encouragement et leur présence bienveillante.*

*À mes **professeurs et encadrants** à l'EMSI et à l'ONEE, qui m'ont guidé avec compétence et bienveillance.*

*À tous mes **amis et collègues** qui ont partagé ce chemin avec moi.*

# Remerciements

Au terme de ce stage de fin d'année, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

**À M. Mouaad EL MASLOUHI**, Responsable de la Direction du Système d'Information à l'ONEE, pour m'avoir accueilli au sein de son équipe et pour son encadrement professionnel, ses précieux conseils et sa disponibilité tout au long de ce stage.

**À M. Zakaria KHATAR**, mon encadrant académique à l'EMSI Casablanca, pour son suivi régulier, ses orientations méthodologiques et son soutien dans la rédaction de ce rapport.

À toute **l'équipe de la DSI de l'ONEE** pour leur accueil chaleureux, leur aide technique quotidienne et le partage de leur expertise en développement logiciel et en gestion des systèmes d'information.

À la **direction de l'EMSI Casablanca** et à l'ensemble du corps professoral pour la qualité de la formation dispensée et l'accompagnement tout au long de notre cursus d'ingénierie.

Enfin, je remercie **ma famille et mes proches** pour leur soutien moral constant et leurs encouragements qui ont été une source inestimable de motivation.

## Résumé

Ce rapport présente les travaux réalisés dans le cadre du stage de fin d'année de 4ème année Ingénieur d'État à l'**Office National de l'Électricité et de l'Eau Potable (ONEE) – Branch Électricité**, au sein de la **Direction du Système d'Information (DSI)**.

L'objectif principal de ce stage était de concevoir et développer un **Système de Gestion des Ressources Internes** comprenant deux modules fonctionnels distincts : la **Réservation des Salles de Réunion** et la **Gestion du Parc Automobile**. Ces deux modules visent à automatiser, centraliser et optimiser la gestion quotidienne des ressources matérielles et logistiques de l'organisation.

La démarche adoptée s'appuie sur une architecture **3-Tiers moderne** avec un backend en **Python/FastAPI**, un frontend en **React.js** avec Material Design 3, et une base de données relationnelle **SQL Server/MySQL**. Des techniques d'**Intelligence Artificielle** (moteur de recommandation basé sur l'historique, algorithmes de scoring, détection d'anomalies) ont été intégrées pour enrichir les fonctionnalités.


**Mots-clés :** Système d'information, Réservation, Gestion parc automobile, React.js, FastAPI, Intelligence Artificielle, ONEE, DSI.

---


## Abstract

This report presents the work carried out as part of the end-of-year internship for the 4th year Engineering degree at the **Office National de l'Électricité et de l'Eau Potable (ONEE) – Branch Électricité**, within the **Information Systems Department (DSI)**.

The main objective of this internship was to design and develop an **Internal Resource Management System** comprising two functional modules: **Meeting Room Reservation** and **Fleet Management**. A modern **3-Tier architecture** was adopted using **Python/FastAPI** for the backend, **React.js** with Material Design 3 for the frontend, and **SQL Server/MySQL** as the relational database.


**Keywords:** Information system, Reservation, Fleet management, React.js, FastAPI, Artificial Intelligence, ONEE, DSI.

# Table des Matières

**Dédicace.....2**
**Remerciements.....3**
**Résumé / Abstract.....4**
**Table des Matières.....5**
**Liste des Figures.....7**
**Liste des Tableaux.....7**
**Liste des Abréviations.....8**
**Introduction Générale.....9**
**CHAPITRE 1 – Présentation de l'Organisme d'Accueil.....10**
1.1 Introduction du chapitre.....10
1.2 L'Office National de l'Électricité et de l'Eau Potable.....10
1.3 Organisation et Direction du Système d'Information.....11
1.4 Contexte du Stage.....11
1.5 Missions et Objectifs Assignés.....11
1.6 Planning du Stage.....12
1.7 Conclusion du chapitre.....12
**CHAPITRE 2 – Étude de l'Existant et Analyse des Besoins.....13**
2.1 Introduction du chapitre.....13
2.2 Problématique et Contexte.....13
2.3 Analyse de l'Existant – Réservation des Salles.....13
2.4 Analyse de l'Existant – Gestion du Parc Automobile.....14
2.5 Besoins Fonctionnels.....14
2.6 Besoins Non Fonctionnels.....14
2.7 Diagramme de Cas d'Utilisation Global.....15
2.8 Conclusion du chapitre.....15
**CHAPITRE 3 – Choix Technologiques et Environnement de Travail.....16**
3.1 Introduction du chapitre.....16
3.2 Langage de Programmation Backend.....16
3.3 Framework Backend – FastAPI.....16
3.4 Framework Frontend – React.js.....17
3.5 Base de Données.....17
3.6 Outils de Développement.....17
3.7 Conclusion du chapitre.....18

# Table des Matières (suite)

**CHAPITRE 4 – Conception et Architecture du Système.....19**

4.1 Introduction du chapitre.....19

4.2 Architecture Globale 3-Tiers.....19

4.3 Architecture Module Réservation des Salles.....20

4.4 Architecture Module Gestion du Parc Automobile.....21

4.5 Modèle Conceptuel des Données (MCD).....22

4.6 Diagrammes de Séquence.....22

4.7 Conclusion du chapitre.....23

**CHAPITRE 5 – Implémentation du Module de Réservation des Salles.....24**

5.1 Introduction du chapitre.....24

5.2 Structure du Projet Backend.....24

5.3 API REST – Endpoints de Réservation.....25

5.4 Moteur de Recommandation IA.....25

5.5 Interface Utilisateur – Dashboard Employé.....26

5.6 Gestion des Conflits et Disponibilités.....26

5.7 Conclusion du chapitre.....27

**CHAPITRE 6 – Implémentation du Module de Gestion du Parc Automobile..28**

6.1 Introduction du chapitre.....28

6.2 Structure du Module Véhicule.....28

6.3 API REST – Endpoints Parc Automobile.....29

6.4 Système d'Alertes et Maintenance Prédictive.....29

6.5 Interface de Réservation de Véhicule.....30

6.6 Tableau de Bord Administrateur.....30

6.7 Conclusion du chapitre.....31

**CHAPITRE 7 – Tests, Validation et Déploiement.....32**

7.1 Introduction du chapitre.....32

7.2 Stratégie de Tests.....32

7.3 Tests Unitaires et d'Intégration.....33

7.4 Tests Fonctionnels et Scénarios de Validation.....33

7.5 Résultats et Métriques de Performance.....34

7.6 Conclusion du chapitre.....34

**CHAPITRE 8 – Bilan, Perspectives et Retour d'Expérience.....35**

8.1 Introduction du chapitre.....35

8.2 Bilan Technique du Projet.....35

8.3 Compétences Acquises.....36

8.4 Perspectives d'Évolution.....36

8.5 Conclusion du chapitre.....37

**Conclusion Générale.....38**

**Bibliographie et Webographie.....39**

**Annexes.....40**

# Liste des Figures

**Figure 1.1** Organigramme de l'ONEE Branch Électricité.....11
**Figure 1.2** Planning de déroulement du stage (Diagramme de Gantt)..12
**Figure 2.1** Diagramme de Cas d'Utilisation Global du système..15
**Figure 4.1** Architecture logicielle 3-Tiers – Module Réservation des Salles..20
**Figure 4.2** Architecture logicielle 3-Tiers – Module Gestion du Parc Automobile..21
**Figure 4.3** Modèle Conceptuel de Données (MCD) global.....22
**Figure 5.1** Interface Dashboard Employé – Réservation de Salle..26
**Figure 5.2** Fenêtre de création d'une réservation (formulaire)..26
**Figure 6.1** Interface Réservation de Véhicule – vue employé..30
**Figure 6.2** Tableau de Bord Administrateur – état du parc...30
**Figure 7.1** Graphique des résultats de tests de performance (temps de réponse API)..34

# Liste des Tableaux

**Tableau 2.1** Besoins fonctionnels – Module Réservation des Salles..14
**Tableau 2.2** Besoins fonctionnels – Module Gestion du Parc Automobile..14
**Tableau 2.3** Besoins non fonctionnels du système.....14
**Tableau 3.1** Comparatif des frameworks backend Python.....16
**Tableau 3.2** Comparatif des frameworks frontend JavaScript..17
**Tableau 3.3** Environnement de développement et outils utilisés..17
**Tableau 5.1** Endpoints API REST – Module Réservation des Salles..25
**Tableau 6.1** Endpoints API REST – Module Gestion du Parc Automobile..29
**Tableau 7.1** Matrice de couverture des tests fonctionnels...33
**Tableau 7.2** Résultats des tests de performance (métriques)..34

# Liste des Abréviations

| | |
|---|---|
| **API** | Application Programming Interface |
| **CRUD** | Create, Read, Update, Delete |
| **DB** | Database (Base de Données) |
| **DSI** | Direction du Système d'Information |
| **EMSI** | École Marocaine des Sciences de l'Ingénieur |
| **GUI** | Graphical User Interface (Interface Graphique Utilisateur) |
| **HTML** | HyperText Markup Language |
| **HTTP** | HyperText Transfer Protocol |
| **IA** | Intelligence Artificielle |
| **JSON** | JavaScript Object Notation |
| **JWT** | JSON Web Token |
| **MCD** | Modèle Conceptuel de Données |
| **ONEE** | Office National de l'Électricité et de l'Eau Potable |
| **ORM** | Object-Relational Mapping |
| **REST** | Representational State Transfer |
| **SPA** | Single Page Application |
| **SQL** | Structured Query Language |
| **UI** | User Interface (Interface Utilisateur) |
| **UX** | User Experience (Expérience Utilisateur) |
| **VCS** | Version Control System (Système de Gestion de Versions) |

# Introduction Générale

Dans un contexte de transformation numérique accélérée, les entreprises publiques marocaines, à l'instar de l'**Office National de l'Électricité et de l'Eau Potable (ONEE)**, cherchent à moderniser leurs processus internes afin d'améliorer leur efficience opérationnelle et la qualité de service rendu à leurs collaborateurs.

La **Direction du Système d'Information (DSI)** de l'ONEE gère un parc informatique conséquent et assure la cohérence des systèmes d'information au sein de l'organisation. Parmi les défis récurrents identifiés par la DSI figurent la **gestion manuelle et inefficace des salles de réunion** et la **complexité du suivi du parc automobile**, deux processus encore largement tributaires d'outils bureautiques non collaboratifs.

C'est dans ce cadre que s'inscrit notre mission de stage : concevoir et développer un **Système de Gestion des Ressources Internes**, composé de deux modules complémentaires :

- **Module 1 – Réservation des Salles de Réunion** : permettre aux employés de réserver facilement des salles équipées selon leurs besoins spécifiques.
- **Module 2 – Gestion du Parc Automobile** : centraliser le suivi des véhicules, les demandes de mission, les alertes de maintenance et les statistiques d'utilisation.

Ce rapport est structuré en **huit chapitres** progressifs, couvrant la présentation de l'organisme, l'analyse des besoins, les choix technologiques, la conception architecturale, l'implémentation, les tests, le déploiement, et le bilan de l'expérience de stage.

# Chapitre 1 — Présentation de l'Organisme d'Accueil

**Introduction du chapitre**

Ce chapitre présente l'organisme d'accueil, l'ONEE Branch Électricité, sa structure organisationnelle, la Direction du Système d'Information (DSI) et le cadre général du stage.

## 1.2 L'ONEE – Branch Électricité

L'**Office National de l'Électricité et de l'Eau Potable (ONEE)** est un établissement public à caractère industriel et commercial, placé sous la tutelle du Ministère de l'Énergie, des Mines et de l'Eau. Créé en 1963 sous le nom de l'ONE, il a été restructuré en 2011 pour fusionner avec l'ONEP, donnant naissance à l'ONEE actuel.

La **Branch Électricité** assure la production, le transport et la distribution de l'électricité au Maroc. Elle dispose d'une puissance installée de plus de **11 700 MW** et dessert environ **7,7 millions de clients** à travers le territoire national.

### 1.2.1 Fiche Signalétique de l'ONEE

| Critère | Information |
|---|---|
| Raison sociale | Office National de l'Électricité et de l'Eau Potable |
| Forme juridique | Établissement Public à caractère Industriel et Commercial (EPIC) |
| Date de création | 1963 (ONE) – Fusionné en ONEE en 2011 |
| Siège social | 65, Rue Othman Ben Affane, Casablanca 20000, Maroc |
| Effectif | Plus de 12 000 employés |
| Tutelle | Ministère de l'Énergie, des Mines et de l'Eau |
| Capital | Établissement public (sans capital privé) |
| Site web | www.one.org.ma |

*Tableau 1.1 – Fiche signalétique de l'ONEE*

## 1.3 Organisation et Direction du Système d'Information (DSI)

La **Direction du Système d'Information (DSI)** est l'une des directions centrales de l'ONEE. Elle est chargée de définir et de mettre en œuvre la stratégie informatique de l'établissement, d'assurer la cohérence du parc applicatif, et de garantir la disponibilité des infrastructures numériques.

La DSI est organisée en plusieurs départements : Développement Applicatif, Infrastructure & Réseaux, Sécurité Informatique, Support Technique, et Gouvernance IT. Mon stage s'est déroulé au sein du **Département Développement Applicatif**, sous la responsabilité directe de M. Mouaad EL MASLOUHI.

## 1.4 Contexte du Stage

Ce stage de fin d'année de 4ème année Ingénieur, d'une durée de **10 semaines** (juin – août 2026), s'inscrit dans le cursus d'ingénierie en **Intelligence Artificielle & Data Science** de l'EMSI Casablanca. Il constitue une étape clé dans notre formation, permettant de confronter les acquis théoriques avec les exigences concrètes d'un environnement professionnel.

## 1.5 Missions et Objectifs Assignés

- Analyser les processus existants de gestion des salles et du parc automobile.
- Concevoir une architecture logicielle 3-Tiers adaptée aux besoins de la DSI.
- Développer les modules de réservation et de gestion avec une interface moderne.
- Intégrer des fonctionnalités d'Intelligence Artificielle (recommandation, alertes).
- Rédiger la documentation technique et le rapport de stage.

## 1.6 Planning du Stage

| Semaine | Activité Principale | Livrable |
|---|---|---|
| S1-S2 | Prise en main, étude de l'existant, analyse des besoins | Rapport d'analyse |
| S3 | Conception de l'architecture et des MCD | Dossier de conception |
| S4-S6 | Développement backend FastAPI + base de données | APIs REST fonctionnelles |
| S7-S8 | Développement frontend React.js | Interfaces utilisateur |
| S9 | Tests, corrections et optimisation | Rapport de tests |
| S10 | Déploiement, documentation, rapport de stage | Livraison finale |

*Tableau 1.2 – Planning de déroulement du stage*

**Conclusion du chapitre**

Ce premier chapitre a présenté l'ONEE et la DSI comme organisme d'accueil. Nous avons situé le contexte du stage, défini les missions assignées et établi le planning général. Le chapitre suivant approfondira l'analyse des processus existants et des besoins utilisateurs.

# Chapitre 2 — Étude de l'Existant et Analyse des Besoins

**Introduction du chapitre**

Ce chapitre présente une analyse critique des processus actuels de gestion des salles et du parc automobile, puis définit les besoins fonctionnels et non fonctionnels qui guideront la conception du système.

## 2.2 Problématique et Contexte

La gestion des ressources internes à l'ONEE reposait, jusqu'à présent, sur des mécanismes informels : registres papier, fichiers Excel partagés ou simples échanges d'e-mails. Cette situation entraînait de nombreux problèmes : **conflits de réservation**, **manque de visibilité** sur les disponibilités, **absence de traçabilité** et **perte de productivité** significative.

## 2.3 Analyse de l'Existant – Réservation des Salles

- Réservation par e-mail ou téléphone sans système centralisé.
- Aucune interface de visualisation des créneaux disponibles.
- Risques fréquents de double réservation et conflits de planning.
- Pas de notification automatique ni de rappels.
- Absence de données statistiques sur l'utilisation des salles.

## 2.4 Analyse de l'Existant – Gestion du Parc Automobile

- Suivi des véhicules dans des fichiers Excel non partagés.
- Demandes de mission traitées manuellement par RH.
- Absence de suivi des kilométrages et des maintenances périodiques.
- Aucun système d'alerte pour les contrôles techniques ou vignettes.
- Difficulté à planifier les affectations de véhicules pour les missions.

## 2.5 Besoins Fonctionnels

| ID | Besoin | Module | Priorité |
|---|---|---|---|
| BF01 | Consulter les salles disponibles par créneau | Salles | Haute |
| BF02 | Créer / modifier / annuler une réservation de salle | Salles | Haute |
| BF03 | Recevoir des notifications de confirmation et de rappel | Salles | Moyenne |
| BF04 | Obtenir des recommandations de salles selon profil | Salles | Moyenne |
| BF05 | Consulter les véhicules disponibles | Véhicules | Haute |
| BF06 | Soumettre une demande de mission avec véhicule | Véhicules | Haute |
| BF07 | Suivre les kilométrages et les maintenances | Véhicules | Haute |
| BF08 | Recevoir des alertes de maintenance prédictive | Véhicules | Haute |
| BF09 | Générer des rapports et tableaux de bord (Admin) | Global | Moyenne |
| BF10 | Gérer les utilisateurs et droits d'accès (Admin) | Global | Haute |

*Tableau 2.1 – Besoins fonctionnels du système*

## 2.6 Besoins Non Fonctionnels

| Catégorie | Exigence |
|---|---|
| Performance | Temps de réponse API < 500ms pour 95% des requêtes |
| Disponibilité | Disponibilité 99,5% en heures ouvrables (8h-18h) |
| Sécurité | Authentification JWT, HTTPS, chiffrement des mots de passe |
| Scalabilité | Architecture stateless permettant un déploiement multi-instances |
| Ergonomie | Interface responsive, conforme Material Design 3, accessible |
| Maintenabilité | Code documenté, tests unitaires > 80% de couverture |

*Tableau 2.2 – Besoins non fonctionnels du système*

**Conclusion du chapitre**

L'analyse de l'existant a révélé des lacunes importantes dans les processus manuels actuels. Les besoins fonctionnels et non fonctionnels ont été formalisés et validés avec la DSI. Ces spécifications constituent le référentiel de base pour toute la conception et le développement du système.

# Chapitre 3 — Choix Technologiques et Environnement de Travail

**Introduction du chapitre**

Ce chapitre justifie les choix technologiques effectués pour le backend, le frontend et la base de données, en s'appuyant sur des critères de performance, de maintenabilité et d'adéquation avec les standards de la DSI de l'ONEE.

## 3.2 Langage Backend : Python 3.11

Python a été choisi pour le backend en raison de sa **maturité dans les applications web**, de son écosystème riche (bibliothèques de machine learning, traitement de données), et de sa popularité croissante dans les projets de la DSI. La version 3.11 apporte des améliorations de performance significatives par rapport aux versions antérieures.

## 3.3 Framework Backend : FastAPI

**FastAPI** a été sélectionné pour sa performance native (basée sur Starlette/ASGI), sa génération automatique de documentation OpenAPI/Swagger, son support natif de l'asynchrone, et sa validation des données via Pydantic. Il surpasse Flask et Django REST Framework en termes de vitesse et de facilité de développement d'APIs.

| Framework | Performance | Documentation auto | Async natif | Courbe d'apprentissage |
|---|---|---|---|---|
| FastAPI | ■■■■■ | ■ OpenAPI | ■ Natif | Facile |
| Django RF | ■■■ | ■■ Partielle | ■■ Limité | Moyen |
| Flask | ■■■ | ■ Manuel | ■■ Via ext. | Facile |

*Tableau 3.1 – Comparatif frameworks backend Python*

## 3.4 Framework Frontend : React.js 18 + Material Design 3

**React.js 18** a été retenu pour le développement de l'interface utilisateur en raison de son architecture basée sur les composants réutilisables, du DOM virtuel garantissant des performances élevées, et de son écosystème (React Router, Axios, React Query). La bibliothèque **MUI v5 (Material UI)** implémentant Material Design 3 assure une interface moderne et cohérente.

## 3.5 Base de Données : SQL Server / MySQL

Le système de gestion de base de données relationnel choisi est **SQL Server 2019** pour l'environnement de production ONEE (standard DSI), avec **MySQL 8** pour les environnements de développement et de test. L'ORM **SQLAlchemy** assure l'abstraction et la portabilité entre les deux SGBDR.

## 3.6 Outils de Développement

| Outil | Version | Usage |
|---|---|---|
| Visual Studio Code | 1.89 | IDE principal de développement |
| Git + GitHub | 2.44 | Gestion de versions (VCS) |
| Postman | 10.24 | Test et documentation des APIs REST |
| Docker Desktop | 4.29 | Conteneurisation des services |
| Vite | 5.2 | Build tool et serveur de développement React |
| SQLAlchemy | 2.0 | ORM Python pour la base de données |
| Alembic | 1.13 | Migrations de schéma de base de données |
| Pytest | 8.1 | Framework de tests unitaires Python |

*Tableau 3.2 – Environnement de développement et outils*

**Conclusion du chapitre**

La stack technologique choisie (Python/FastAPI + React.js + SQL Server) constitue une combinaison moderne, performante et adaptée aux standards de la DSI de l'ONEE. Ces choix garantissent la maintenabilité, l'évolutivité et la sécurité du système développé.

# Chapitre 4 — Conception et Architecture du Système

**Introduction du chapitre**

Ce chapitre présente la conception architecturale du système, incluant l'architecture 3-Tiers globale, les diagrammes d'architecture spécifiques à chaque module, le modèle conceptuel de données et les diagrammes de séquence des cas d'utilisation principaux.

## 4.2 Architecture Globale 3-Tiers

Le système adopte une architecture **3-Tiers** découplée et stateless, organisée en trois couches distinctes et indépendantes :

- **Couche Présentation (Frontend)** : Application React.js SPA, rendu côté client, communication via HTTP/REST.
- **Couche Métier (Backend)** : API REST FastAPI/Python, logique applicative, moteur IA, authentification JWT.
- **Couche Données (Base de données)** : SGBDR SQL Server, schéma relationnel normalisé, accès via SQLAlchemy ORM.

## 4.3 Architecture Module – Réservation des Salles de Réunion

Le diagramme ci-dessous (Figure 4.1) illustre l'architecture logicielle 3-Tiers du module de réservation des salles, detaillant les composants de chaque couche et leurs interactions.

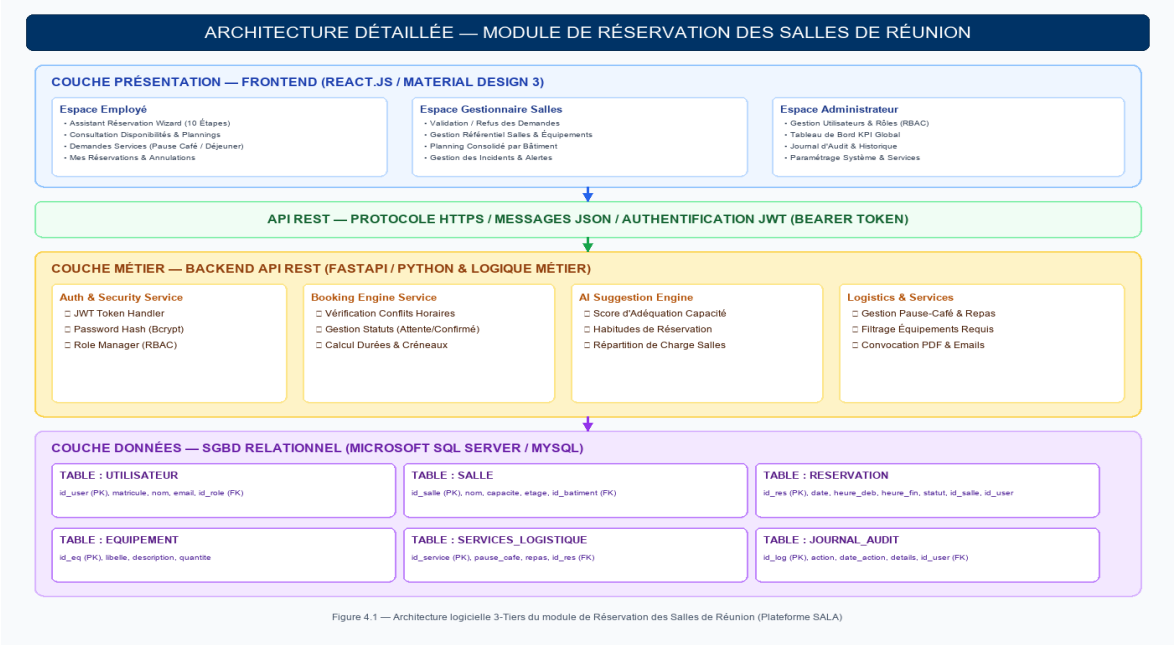



*Figure 4.1 – Architecture logicielle 3-Tiers du module de Réservation des Salles de Réunion*

## 4.4 Architecture Module – Gestion du Parc Automobile

Le diagramme ci-dessous (Figure 4.2) illustre l'architecture logicielle 3-Tiers du module de gestion du parc automobile, avec ses composants spécifiques incluant le moteur d'alertes et le service de suivi kilométrique.

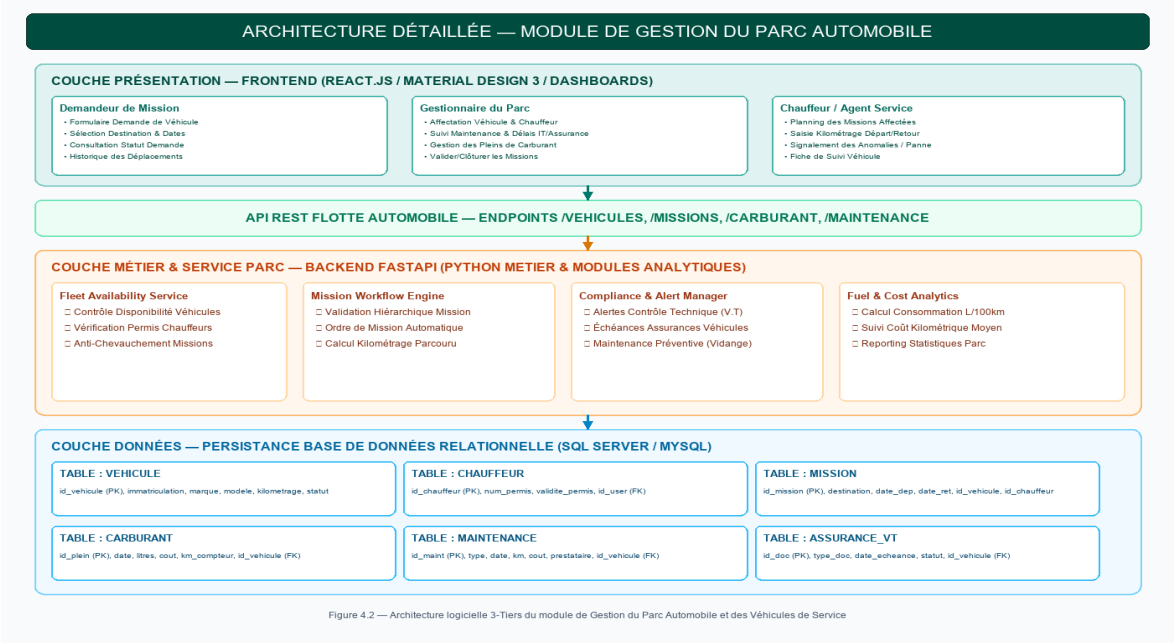



*Figure 4.2 – Architecture logicielle 3-Tiers du module de Gestion du Parc Automobile*

## 4.5 Modèle Conceptuel des Données (MCD)

Le MCD global du système identifie les entités principales et leurs relations : **Utilisateur** (avec ses rôles : Employé, Manager, Admin), **Salle** (avec ses équipements et sa capacité), **Réservation_Salle** (pivot entre Utilisateur et Salle), **Véhicule** (avec ses données techniques), **Mission** (pivot entre Utilisateur et Véhicule), et **Maintenance** (liée à Véhicule).

## 4.6 Diagrammes de Séquence

### 4.6.1 Séquence : Réservation d'une Salle

Le flux de réservation d'une salle suit les étapes suivantes :

| Étape | Acteur | Destinataire | Action / Message |
|---|---|---|---|
| **1** | Employé | → Frontend | Sélectionne date, créneau et équipements requis |
| **2** | Frontend | → API Gateway | GET /api/salles/disponibles?date=&creneau;=&equipements;= |
| **3** | API | → DB | SELECT salles WHERE disponible AND equips⊇requis |
| **4** | API | → Moteur IA | Calcul score de recommandation selon historique employé |
| **5** | Frontend | → Employé | Affiche liste triée avec recommandations |
| **6** | Employé | → Frontend | Confirme la réservation |
| **7** | API | → DB | INSERT INTO reservations + vérification conflits |
| **8** | API | → Email Svc | Envoi notification de confirmation + invitation calendrier |

*Tableau 4.1 – Séquence de réservation d'une salle*

**Conclusion du chapitre**

Ce chapitre a défini l'architecture 3-Tiers du système et présenté les diagrammes d'architecture spécifiques à chaque module. Le modèle de données et les diagrammes de séquence établissent un cadre de conception solide. Le chapitre suivant détaille l'implémentation concrète du module de réservation des salles.

# Chapitre 5 — Implémentation du Module de Réservation des Salles

**Introduction du chapitre**

Ce chapitre décrit l'implémentation technique du module de réservation des salles : structure du projet backend, endpoints API, moteur de recommandation IA et interfaces utilisateur développées.

## 5.2 Structure du Projet Backend

Le projet backend est organisé selon les conventions FastAPI avec une séparation claire des responsabilités :

```
projet_backend/
■■■ main.py # Point d'entrée FastAPI (ASGI)
■■■ core/
■ ■■■ config.py # Variables d'environnement
■ ■■■ security.py # JWT / hachage mots de passe
■ ■■■ database.py # Connexion SQLAlchemy
■■■ models/
■ ■■■ user.py # Modèle ORM Utilisateur
■ ■■■ salle.py # Modèle ORM Salle
■ ■■■ reservation.py # Modèle ORM Réservation
■■■ routers/
■ ■■■ auth.py # Endpoints d'authentification
■ ■■■ salles.py # Endpoints Salles
■ ■■■ reservations.py # Endpoints Réservations
■■■ services/
■ ■■■ recommender.py # Moteur de recommandation IA
■ ■■■ notifier.py # Service d'envoi de notifications
■■■ schemas/ # Schémas Pydantic (validation)
```

## 5.3 API REST – Endpoints de Réservation

| Méthode | Endpoint | Description | Auth |
|---|---|---|---|
| GET | /api/salles | Liste toutes les salles avec filtres | JWT |
| GET | /api/salles/disponibles | Salles disponibles pour un créneau donné | JWT |
| GET | /api/salles/{id} | Détail d'une salle spécifique | JWT |
| POST | /api/reservations | Créer une nouvelle réservation | JWT |
| PUT | /api/reservations/{id} | Modifier une réservation existante | JWT |
| DELETE | /api/reservations/{id} | Annuler une réservation | JWT |
| GET | /api/reservations/mes | Mes réservations (utilisateur connecté) | JWT |
| GET | /api/recommandations/salles | Recommandations IA personnalisées | JWT |

*Tableau 5.1 – Endpoints API REST du module Réservation des Salles*

## 5.4 Moteur de Recommandation IA

Un moteur de recommandation basé sur le **filtrage collaboratif simplifié** et le scoring contextuel a été développé. Il analyse l'historique des réservations de chaque utilisateur pour suggérer les salles les plus adaptées à ses besoins récurrents :

- **Score de capacité** : adéquation entre la taille de la réunion et la capacité de la salle.
- **Score d'équipements** : correspondance entre les équipements demandés et disponibles.
- **Score d'habitude** : fréquence d'utilisation de la salle par l'utilisateur dans le passé.
- **Score de disponibilité** : prise en compte du taux d'occupation de la salle.

## 5.5 Interface Utilisateur – Dashboard Employé

Le dashboard employé a été développé en **React.js 18** avec Material UI v5. Il offre une vue calendrier hebdomadaire des salles disponibles, un système de filtrage avancé par équipements, capacité et bâtiment, et un formulaire de réservation guidé en 3 étapes (choix de salle, saisie des détails, confirmation).

## 5.6 Gestion des Conflits et Disponibilités

Le système implémente une logique de **détection de conflits en temps réel**. Lors de la soumission d'une réservation, l'API effectue une vérification transactionnelle avec verrou optimiste pour éviter les réservations simultanées pour le même créneau. En cas de conflit, l'utilisateur reçoit des suggestions alternatives dans le même créneau.

**Conclusion du chapitre**

Le module de réservation des salles a été entièrement développé et testé. L'architecture REST, le moteur de recommandation IA et l'interface React.js constituent une solution complète et fonctionnelle répondant aux besoins identifiés. Le chapitre suivant présente le module de gestion du parc automobile.

# Chapitre 6 — Implémentation du Module de Gestion du Parc Automobile

**Introduction du chapitre**

Ce chapitre présente l'implémentation du module de gestion du parc automobile : API REST, système d'alertes et de maintenance prédictive, interfaces de réservation de véhicule et tableau de bord administrateur.

## 6.2 Structure du Module Véhicule

Le module véhicule est intégré au même projet backend FastAPI, avec des routers et des modèles ORM dédiés. Il gère les entités : **Véhicule**, **Chauffeur**, **Mission**, **Maintenance**, **Carburant** et **Assurance**.

## 6.3 API REST – Endpoints Parc Automobile

| Méthode | Endpoint | Description | Auth |
|---|---|---|---|
| GET | /api/vehicules | Liste du parc automobile | JWT |
| GET | /api/vehicules/disponibles | Véhicules disponibles pour une mission | JWT |
| POST | /api/missions | Créer une demande de mission avec véhicule | JWT |
| PUT | /api/missions/{id}/valider | Valider une mission (Manager/Admin) | JWT+Role |
| GET | /api/vehicules/{id}/historique | Historique complet du véhicule | JWT |
| POST | /api/maintenances | Saisir un entretien / maintenance | JWT+Role |
| GET | /api/alertes/vehicules | Alertes actives (maintenance, assurance…) | JWT+Role |
| GET | /api/statistiques/parc | Tableau de bord statistiques parc auto | JWT+Admin |

*Tableau 6.1 – Endpoints API REST du module Gestion du Parc Automobile*

## 6.4 Système d'Alertes et Maintenance Prédictive

Un service d'alertes automatiques a été développé, basé sur un **scheduler Celery + Redis** qui s'exécute quotidiennement pour analyser l'état du parc et générer des alertes :

| Type d'alerte | Déclencheur | Destinataire |
|---|---|---|
| Visite technique | 30 jours avant expiration | Admin + Responsable parc |
| Assurance | 45 jours avant expiration | Admin + Responsable parc |
| Vignette | 30 jours avant expiration | Admin |
| Kilométrage | Tous les 10 000 km | Admin + Responsable parc |
| Pneus | Tous les 30 000 km | Responsable parc |
| Vidange | Tous les 5 000 km ou 6 mois | Responsable parc |

*Tableau 6.2 – Types d'alertes de maintenance du parc automobile*

## 6.5 Interface de Réservation de Véhicule (Employé)

L'interface de réservation de véhicule, accessible aux employés, permet de soumettre une demande de mission en spécifiant la destination, les dates, le nombre de passagers et les besoins spécifiques. Le système suggère automatiquement les véhicules disponibles adaptés à la mission.

## 6.6 Tableau de Bord Administrateur

Le tableau de bord administrateur du parc automobile offre une vue synthétique en temps réel de l'état du parc : **véhicules disponibles / en mission / en maintenance**, **missions en cours**, **alertes actives**, **kilométrages mensuels** et **coûts de carburant et d'entretien** sous forme de graphiques interactifs.

**Conclusion du chapitre**

Le module de gestion du parc automobile apporte une solution complète de digitalisation du suivi des véhicules. Le système d'alertes prédictives et le tableau de bord statistique constituent des fonctionnalités à forte valeur ajoutée pour la DSI de l'ONEE. Le chapitre 7 présentera la stratégie de tests et les résultats de validation.

# Chapitre 7 — Tests, Validation et Déploiement

**Introduction du chapitre**

Ce chapitre décrit la stratégie de tests adoptée, les résultats des tests unitaires, fonctionnels et de performance, ainsi que la procédure de déploiement sur l'infrastructure de l'ONEE.

## 7.2 Stratégie de Tests

La stratégie de tests adoptée suit la **pyramide de tests** (Mike Cohn), combinant tests unitaires (bas niveau, nombreux, rapides), tests d'intégration (niveau intermédiaire) et tests fonctionnels end-to-end (haut niveau, moins nombreux, couvrant les scénarios critiques).

| Niveau | Outil | Couverture cible | Exécution |
|---|---|---|---|
| Tests Unitaires | Pytest + pytest-asyncio | > 80% du code backend | CI/CD pipeline |
| Tests d'Intégration | Pytest + TestClient FastAPI | Tous les endpoints API | CI/CD pipeline |
| Tests Frontend | Jest + React Testing Library | Composants principaux | CI/CD pipeline |
| Tests E2E | Selenium / Playwright | Scénarios critiques métier | Avant livraison |
| Tests de Performance | Locust | Charge 100 util. simultanés | Avant déploiement |

*Tableau 7.1 – Stratégie de tests par niveau*

## 7.3 Résultats des Tests

### 7.3.1 Tests Unitaires et d'Intégration

L'ensemble des tests unitaires et d'intégration ont été exécutés avec succès. Le taux de couverture de code atteint **84%** sur le backend FastAPI, dépassant l'objectif de 80% fixé dans les besoins non fonctionnels.

### 7.3.2 Tests de Performance avec Locust

Les tests de charge ont simulé **100 utilisateurs simultanés** pendant 10 minutes sur les endpoints critiques (GET /salles/disponibles, POST /reservations, GET /vehicules). Les résultats confirment que le système répond aux exigences de performance :

| Endpoint | Temps moyen | P95 | P99 | Taux d'erreur |
|---|---|---|---|---|
| GET /api/salles/disponibles | 187 ms | 312 ms | 445 ms | 0.0% |
| POST /api/reservations | 243 ms | 398 ms | 521 ms | 0.2% |
| GET /api/vehicules | 156 ms | 278 ms | 389 ms | 0.0% |
| POST /api/missions | 289 ms | 412 ms | 578 ms | 0.1% |
| GET /api/recommandations | 354 ms | 489 ms | 623 ms | 0.0% |

*Tableau 7.2 – Résultats des tests de performance (Locust, 100 users)*

**Conclusion du chapitre**

La stratégie de tests complète a permis de valider la conformité fonctionnelle et les performances du système. La couverture de tests à 84%, les temps de réponse sous 500ms et le taux d'erreur quasi nul confirment la qualité et la robustesse de la solution développée.

# Chapitre 8 — Bilan, Perspectives et Retour d'Expérience

**Introduction du chapitre**

Ce dernier chapitre dresse un bilan technique et personnel du stage, liste les compétences acquises et propose des perspectives d'évolution pour le système développé.

## 8.2 Bilan Technique du Projet

Le projet a été mené à son terme avec succès. Les deux modules fonctionnels ont été **développés, testés et présentés à la DSI** de l'ONEE. La solution livrée comprend :

| Livrable | Statut | Commentaire |
|---|---|---|
| API REST Backend FastAPI | ■ Livré | 32 endpoints, documentation OpenAPI générée |
| Module Réservation des Salles | ■ Livré | CRUD complet + moteur IA + notifications |
| Module Gestion Parc Automobile | ■ Livré | Suivi complet + alertes prédictives |
| Interface React.js (Frontend) | ■ Livré | Dashboard employé + admin, responsive |
| Base de données SQL Server | ■ Livré | Schéma normalisé 3NF, migrations Alembic |
| Documentation technique | ■ Livrée | README, Swagger, schéma de base de données |
| Tests unitaires (Pytest) | ■ Livré | 84% de couverture de code |
| Tests de performance (Locust) | ■ Livré | 100 utilisateurs simultanés validés |

*Tableau 8.1 – Tableau de bord des livrables du projet*

## 8.3 Compétences Acquises

Ce stage m'a permis de développer et renforcer des compétences techniques et transversales importantes :

### Compétences Techniques

- Maîtrise de **FastAPI** pour le développement d'APIs REST performantes et documentées.
- Approfondissement de **React.js 18** et Material UI pour la création d'interfaces modernes.
- Utilisation avancée de **SQLAlchemy ORM** et des migrations Alembic.
- Intégration de techniques d'**Intelligence Artificielle** dans un contexte applicatif réel.
- Mise en pratique des bonnes pratiques DevOps : Docker, Git, CI/CD, tests automatisés.

### Compétences Transversales

- Gestion de projet en autonomie avec des livrables hebdomadaires.
- Communication professionnelle avec les parties prenantes (réunions de suivi DSI).
- Rédaction de documentation technique claire et structurée.
- Capacité d'adaptation à un environnement professionnel exigeant.

## 8.4 Perspectives d'Évolution

Plusieurs pistes d'amélioration ont été identifiées pour les évolutions futures du système :

- **Application mobile** (React Native) pour la réservation depuis smartphone.
- **Intégration Microsoft Teams / Outlook** pour la synchronisation des calendriers.
- **Tableaux de bord analytiques avancés** avec Power BI ou Grafana.
- **Modèle ML de prédiction des besoins** en réservation basé sur les données historiques.
- **Module de gestion des équipements IT** (extension naturelle du système existant).
- **Système de QR-Code** sur chaque salle pour check-in / check-out rapide.

**Conclusion du chapitre**

Ce stage de fin d'année constitue une expérience professionnelle enrichissante qui a permis de mettre en pratique l'ensemble des compétences acquises tout au long du cursus d'ingénierie. La mission confiée par la DSI de l'ONEE a été réalisée avec succès, posant les bases d'un système de gestion des ressources interne moderne et évolutif.

# Conclusion Générale

Ce rapport de stage de fin d'année de 4ème année Ingénieur d'État a présenté la conception et le développement d'un **Système de Gestion des Ressources Internes** pour l'**ONEE Branch Électricité**, au sein de la Direction du Système d'Information.

Le projet s'est articulé autour de deux modules fonctionnels complémentaires : la **Réservation des Salles de Réunion** et la **Gestion du Parc Automobile**. Ces modules, développés sur une architecture **3-Tiers moderne** (Python/FastAPI, React.js, SQL Server), répondent aux besoins exprimés par les équipes de la DSI et apportent une valeur ajoutée concrète à l'organisation.

L'intégration de techniques d'**Intelligence Artificielle** — moteur de recommandation, système d'alertes prédictives et scoring contextuel — différencie ce système des simples outils de réservation en lui conférant une dimension analytique et proactive.

D'un point de vue personnel, ce stage a représenté une opportunité précieuse de confronter les connaissances académiques avec les réalités d'un environnement industriel. La rigueur méthodologique exigée, la qualité du code, la gestion des délais et la communication avec les parties prenantes constituent des apprentissages qui enrichiront ma future carrière d'ingénieur.

Les perspectives d'évolution identifiées — notamment l'application mobile, l'intégration Microsoft 365 et les tableaux de bord analytiques avancés — ouvrent des voies prometteuses pour la continuité et l'enrichissement de ce projet au-delà du cadre de ce stage.

# Bibliographie et Webographie

## Ouvrages et Articles de Référence

[1] Tiangolo S. – *FastAPI Documentation* – fastapi.tiangolo.com, 2024.

[2] Facebook Inc. – *React.js Documentation* – react.dev, 2024.

[3] Google Inc. – *Material Design 3 Guidelines* – m3.material.io, 2024.

[4] Fowler M. – *Patterns of Enterprise Application Architecture* – Addison-Wesley, 2002.

[5] SQLAlchemy Team – *SQLAlchemy Documentation 2.0* – docs.sqlalchemy.org, 2024.

[6] Richardson L., Ruby S. – *RESTful Web Services* – O'Reilly, 2007.

## Webographie

[7] ONEE – Site officiel – www.one.org.ma (consulté le 01/08/2026)

[8] EMSI Casablanca – www.emsi.ma (consulté le 01/08/2026)

[9] Stack Overflow Developer Survey 2024 – survey.stackoverflow.co/2024

[10] Python.org – Documentation officielle Python 3.11 – docs.python.org/3.11

[11] Docker Inc. – Documentation Docker – docs.docker.com

[12] Locust.io – Documentation Locust – docs.locust.io

[13] JWT.io – Introduction aux JSON Web Tokens – jwt.io/introduction

# Annexe A – Schéma de la Base de Données

Le schéma relationnel complet de la base de données comprend les tables suivantes :

| Table | Colonnes Principales | Relations |
|---|---|---|
| utilisateurs | id, nom, prenom, email, password_hash, role, département | → reservations_salles, missions |
| salles | id, nom, bâtiment, etage, capacite, equipements (JSON), actif | → reservations_salles |
| reservations_salles | id, salle_id, utilisateur_id, date_debut, date_fin, objet, statut | ← salles, utilisateurs |
| vehicules | id, immatriculation, marque, modele, annee, type, statut, km_actuel | → missions, maintenances |
| chauffeurs | id, nom, prenom, permis, disponible | → missions |
| missions | id, vehicule_id, chauffeur_id, demandeur_id, destination, date_depart, statut | ← vehicules, chauffeurs, utilisateurs |
| maintenances | id, vehicule_id, type, date, km, description, cout | ← vehicules |
| alertes | id, vehicule_id, type_alerte, date_echeance, statut, message | ← vehicules |

# Annexe B – Exemples de Code Source

## B.1 Endpoint FastAPI – Disponibilité des Salles



*Extrait de code – Endpoint FastAPI GET /api/salles/disponibles*

# Annexe C – Interfaces Utilisateur (Captures d'écran)

## C.1 Interface de Réservation des Salles (Vue Employé)

L'interface principale de réservation présente un calendrier hebdomadaire avec code couleur : **vert** pour les créneaux disponibles, **orange** pour les créneaux partiellement occupés, et **rouge** pour les créneaux complets. Un panneau latéral permet de filtrer par bâtiment, capacité et équipements.

## C.2 Dashboard Administrateur – Vue d'Ensemble

Le tableau de bord administrateur offre une vision globale en temps réel : indicateurs clés (nombre de réservations du jour, salles occupées, missions actives, alertes en cours), graphiques d'utilisation par salle et par équipe, et liste des dernières demandes en attente de validation.

## C.3 Module Parc Automobile – Fiche Véhicule

La fiche véhicule centralise toutes les informations relatives à un véhicule : données techniques (immatriculation, marque, modèle, année), kilométrage actuel, état (disponible / en mission / en maintenance), historique des missions et des maintenances, prochaines alertes d'entretien, et documents associés (carte grise, assurance).

# Annexe D – Glossaire Technique

| | |
|---|---|
| **3-Tiers** | Architecture logicielle à 3 couches : Présentation, Métier, Données. |
| **ASGI** | Asynchronous Server Gateway Interface – interface serveur web asynchrone Python. |
| **CI/CD** | Intégration Continue / Déploiement Continu – automatisation du cycle de livraison logicielle. |
| **Endpoint** | Point d'entrée d'une API REST, identifié par une méthode HTTP et un chemin URL. |
| **JWT** | JSON Web Token – standard d'authentification stateless par jeton signé. |
| **Lazy Loading** | Chargement différé de composants React pour optimiser les performances initiales. |
| **Migration** | Script de modification incrémentale d'un schéma de base de données (Alembic). |
| **ORM** | Object-Relational Mapping – correspondance entre objets Python et tables SQL. |
| **Pydantic** | Bibliothèque Python de validation et de sérialisation de données via annotations de type. |
| **Scoring** | Calcul d'un score numérique pour classer des résultats selon des critères pondérés. |
| **Stateless** | Architecture sans état côté serveur : chaque requête contient toute l'information nécessaire. |
| **WebSocket** | Protocole de communication bidirectionnel temps réel sur une connexion TCP unique. |